HW 5 - CNN Sketch Classification

Name: Dawud Shakir **Course:** CS591 DIP **Date:** Thursday, April 16

Purpose. This report is a comparative performance analysis for a sketch classifier built on the Google QuickDraw dataset. A baseline neural network is compared against multiple CNN configurations.

1 Dataset and Experimental Setup

Dataset split. For each class, I used 800 training sketches and 200 test sketches. All sketches were resized to **28×28** grayscale images and normalized using the mean and standard deviation of the selected sketches.

Example sketches from each of the five selected classes.

Training details. All neural networks were trained using the same hyperparameters to ensure a fair comparison between configurations.

- Optimizer: Adam
- Learning rate: 0.001
- Batch size: 64
- Epochs: 10
- Loss: Cross Entropy

2 Models

In the baseline model, each input image is flattened from 28×28 to a vector of length 784, passed through a fully connected hidden layer, and then mapped to the output classes.

Why this baseline matters. This model is a reference point for comparison. Because it only uses fully connected layers after flattening, it does not explicitly preserve spatial structure the way a CNN does. I therefore expect the CNN models to perform better, especially on classes with unique local patterns.

3 CNN Configurations Tested

I tested multiple CNN configurations, changing model settings such as the number of convolution layers, hidden sizes, batch normalization, and dropout. CNN-A is the simplest CNN tested, with a single convolution layer followed by a fully connected layer, while CNN-B adds one more convolution layer,

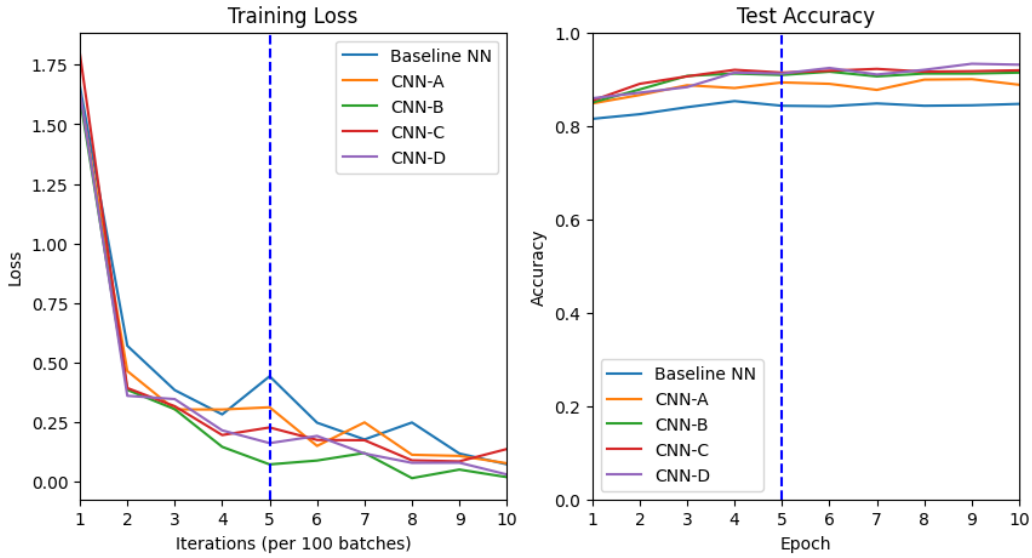
CNN-C adds batch normalization, and CNN-D adds a larger fully connected layer. Table 1 summarizes the compared models.

Model	Architecture Summary	Batch Norm	Dropout	Epochs
Baseline NN	Flatten $\rightarrow$ FC$(784, 128)$ $\rightarrow$ ReLU $\rightarrow$ FC $\rightarrow$ output	No	0.2	10
CNN-A	Conv$(1, 16, 3)$ $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ FC$(16{\cdot}14{\cdot}14 \rightarrow 128)$	No	0.2	10
CNN-B	Conv$(1, 16, 3)$ $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Conv$(16, 32, 3)$ $\rightarrow$ ReLU $\rightarrow$ FC$(32{\cdot}7{\cdot}7 \rightarrow 128)$	No	0.2	10
CNN-C	Conv$(1, 16, 3)$ $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Pool $\rightarrow$ Conv $(16, 32, 3)$ $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Pool $\rightarrow$ FC$(32{\cdot}7{\cdot}7 \rightarrow 128)$	Yes	0.3	10
CNN-D	Conv$(1, 16, 3)$ $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Pool $\rightarrow$ Conv$(16, 32, 3)$ $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Pool $\rightarrow$ FC$(32{\cdot}7{\cdot}7 \rightarrow 256)$	Yes	0.3	10

Table 1: Model configurations compared in this report. All CNN models used a 3×3 kernel size and a 1 padding.

Observation. Adding convolution layers helped because sketch images are sparse and shape-based. Batch normalization improved training stability, while dropout improved generalization when the model started to overfit.

4 Quantitative Results



This figure shows how each model's accuracy and loss evolved over the training epochs, allowing comparison of convergence speed and stability. The Baseline NN shows slower convergence and CNN-A and CNN-C show more fluctuation in loss in training, while CNN-B shows the most stable convergence with minimal fluctuations in loss by the final epoch, indicating effective regularization and training stability.

Table 2 and 3 are the main comparison table for the report.

Model	**Train Acc.**	**Test Acc.**	**Loss Diff. (avg/last)**
Baseline NN	91%	85%	0.373/0.131
CNN-A (single convolution layer)	93%	89%	0.449/0.144
CNN-B (two convolution layers)	95%	91%	0.394/0.060
CNN-C (two convolution layers + batch normalization)	94%	91%	0.364/0.034
CNN-D (two convolution layers + batch normalization + larger FC layer)	98%	92%	0.369/0.054

Table 2: **Epoch 1-5**. The baseline model predicted "star" for most classes and underfit, but began to improve and become more stable by the fifth epoch. CNN-A had better accuracy, but confused "star" and "cat"; moderate stability. CNN-B achieved high accuracy with most predictions correct and improved stability by the fifth epoch. CNN-C achieved high accuracy and the best stability, with predictions mostly correct. CNN-D achieved the highest training accuracy and the second-best loss stability, with predictions mostly correct.

Model	**Train Acc.**	**Test Acc.**	**Loss Diff. (avg/last)**
Baseline NN	96%	86%	0.171/0.028
CNN-A (single convolution layer)	96%	90%	0.202/0.032
CNN-B (two convolution layers)	99%	92%	0.181/0.021
CNN-C (two convolution layers + batch normalization)	97%	92%	0.208/0.028
CNN-D (two convolution layers + batch normalization + larger FC layer)	99%	91%	0.185/0.049

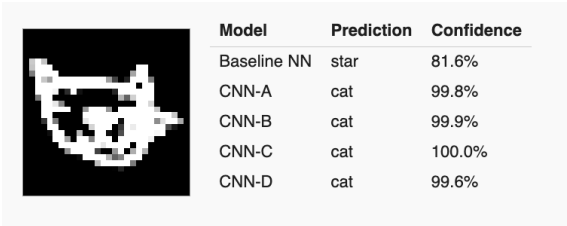
Table 3: **Epoch 6-10**. The baseline NN slightly improved but continues to slightly underfit. CNN-A improves accuracy but "star" remains a common prediction. CNN-B achieves the highest accuracy and stability. CNN-C also achieves high accuracy and stability. CNN-D achieves the highest training accuracy but with a slightly lower test accuracy than CNN-B and CNN-C.

Best configuration. The best-performing model was **CNN-C**, which achieved a test accuracy of 92%. Compared with the baseline NN, this was an improvement of 6 percentage points.

Result summary. In general, the CNN models did not strongly outperform the baseline model. The largest accuracy gain appeared when I added a second convolution layer. Increasing the width of the fully connected layer in CNN-D did not improve test accuracy and slightly increased overfitting.

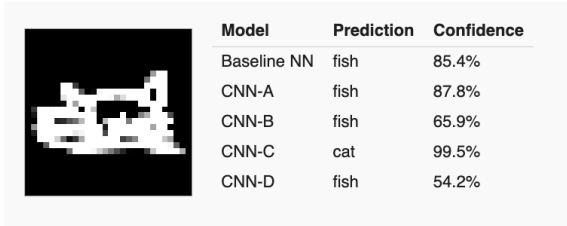
For CNN-A and CNN-B, the loss difference metric shows higher instability at the beginning of training. However, this is expected because the model learns rapidly at first, then stabilizes.

5 Comparative Classification Output Screenshots (Epoch 10)



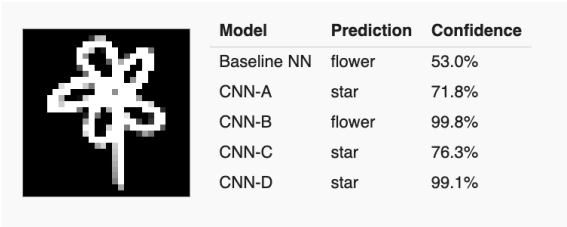
Model	Prediction	Confidence
Baseline NN	star	81.6%
CNN-A	cat	99.8%
CNN-B	cat	99.9%
CNN-C	cat	100.0%
CNN-D	cat	99.6%

Cat



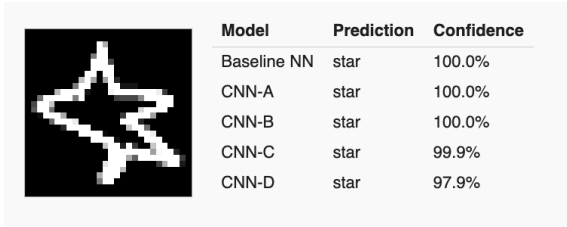
Model	Prediction	Confidence
Baseline NN	fish	85.4%
CNN-A	fish	87.8%
CNN-B	fish	65.9%
CNN-C	cat	99.5%
CNN-D	fish	54.2%

Fish



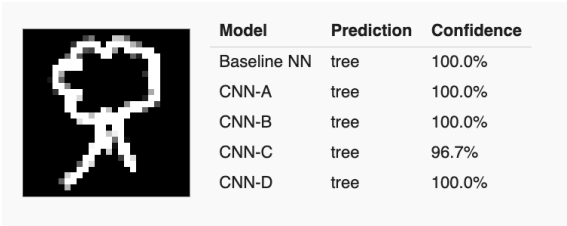
Model	Prediction	Confidence
Baseline NN	flower	53.0%
CNN-A	star	71.8%
CNN-B	flower	99.8%
CNN-C	star	76.3%
CNN-D	star	99.1%

Flower



Model	Prediction	Confidence
Baseline NN	star	100.0%
CNN-A	star	100.0%
CNN-B	star	100.0%
CNN-C	star	99.9%
CNN-D	star	97.9%

Star



Model	Prediction	Confidence
Baseline NN	tree	100.0%
CNN-A	tree	100.0%
CNN-B	tree	100.0%
CNN-C	tree	96.7%
CNN-D	tree	100.0%

Tree

6 Discussion

Why the CNN helped. The CNN improved performance because convolution layers learn local features such as edges, curves, corners, and simple shape parts before combining them into higher-level features. This makes a CNN better suited than a fully connected network for recognizing sketch structure.

Configuration effects. Based on my experiments:

- Increasing model capacity from one convolutional layer to two convolutional layers caused better accuracy.
- Adding batch normalization caused more stable training.
- Adding dropout caused better generalization.
- A weaker configuration was CNN-A, which suggests that having only one convolutional layer was insufficient to capture enough hierarchical features from the sketches to achieve a higher accuracy.

What I would try next. If I had more time, I would retest with more classes and more data per class to see if the CNN could generalize better with a larger and more diverse dataset.

7 Conclusion

This homework compared a fully connected neural network against several CNN configurations on the QuickDraw sketch dataset. The results showed that convolutional models consistently outperformed the baseline, with the best CNNs improving test accuracy from 86% to 92%. Among the tested models, CNN-C provided the best overall balance of accuracy, stability, and generalization, making it the strongest final choice for this experiment.